

by inserting a memory fence (such as the one we saw in Sec. 2.5.9) right after the branch instruction in Line 1, which simply stops all speculation until the condition is resolved.

Transient execution attacks such as Meltdown and Spectre have spawned a whole new domain of security research. New vulnerabilities are found every few months and new fixes are released by CPU vendors and operating system developers at the same frequency. Unfortunately, all these mitigations chip away at performance. You may find that some newer processors (with all defenses up) are slower than older processors. We have not seen that often in the past 50 years!

9.7 INSIDER ATTACKS

A whole different category of attacks are what might be termed “inside jobs.” These are executed by programmers and other employees of the company running the computer to be protected or making critical software. These attacks differ from external attacks because the insiders have specialized knowledge and access that outsiders do not have. Below we will give a few examples; all of them have occurred repeatedly in the past. Each one has a different flavor in terms of who is doing the attacking, who is being attacked, and what the attacker is trying to achieve.

9.7.1 Logic Bombs

In these times of massive outsourcing, programmers often worry about their jobs. Sometimes they even take steps to make their potential (involuntary) departure less painful. For those who are inclined toward blackmail, one strategy is to write a **logic bomb**. This device is a piece of code written by one of a company’s (currently employed) programmers and secretly inserted into the production system. As long as the programmer feeds it its daily password, it is happy and does nothing. However, if the programmer is suddenly fired and physically removed from the premises without warning, the next day (or next week) the logic bomb does not get fed its daily password, so it goes off. Many variants on this theme are also possible. In one famous case, the logic bomb checked the payroll. If the personnel number of the programmer did not appear in it for two consecutive payroll periods, it went off (Spafford et al., 1989).

Going off might involve clearing the disk, erasing files at random, carefully making hard-to-detect changes to key programs, or encrypting essential files. In the latter case, the company has a tough choice about whether to call the police (which may or may not result in a conviction many months later but certainly does not restore the missing files) or to give in to the blackmail and rehire the ex-programmer as a “consultant” for an astronomical sum to fix the problem (and hope that he does not plant new logic bombs while doing so).

There have been recorded cases in which a virus planted a logic bomb on the computers it infected. Generally, these were programmed to go off all at once at some date and time in the future. However, since the programmer has no idea in advance of which computers will be hit, logic bombs planted by viruses cannot be used for job protection or blackmail. Often they are set to go off on a date that has some political significance. Sometimes these are called **time bombs**.

9.7.2 Back Doors

Another security hole caused by an insider is the **back door**. This problem is created by code inserted into the system by a system programmer to bypass some normal check. For example, a programmer could add code to the login program to allow anyone to log in using the login name “zzzzz” no matter what was in the password file. The normal code in the login program might look something like Fig. 9-31(a). The back door would be the change to Fig. 9-31(b).

<pre>while (TRUE) { printf("login: "); get_string(name); disable_echoing(); printf("password: "); get_string(password); enable_echoing(); v = check_validity(name, password); if (v) break; } execute_shell(name);</pre> (a)	<pre>while (TRUE) { printf("login: "); get_string(name); disable_echoing(); printf("password: "); get_string(password); enable_echoing(); v = check_validity(name, password); if (v strcmp(name, "zzzzz") == 0) break; } execute_shell(name);</pre> (b)
---	---

Figure 9-31. (a) Normal code. (b) Code with a back door inserted.

What the call to *strcmp* does is check if the login name is “zzzzz”. If so, the login succeeds, no matter what password is typed. If this back-door code were inserted by a programmer working for a computer manufacturer and then shipped with its computers, the programmer could log into any computer made by his company, no matter who owned it or what was in the password file. The same holds for a programmer working for the OS vendor. The back door simply bypasses the whole authentication process.

One way for companies to prevent backdoors is to have **code reviews** as standard practice. With this technique, once a programmer has finished writing and testing a module, the module is checked into a code database. Periodically, all the programmers in a team get together and each one gets up in front of the group to explain what his code does, line by line. Not only does this greatly increase the chance that someone will catch a back door, but it raises the stakes for the programmer, since being caught red-handed is probably not a plus for his career. If

the programmers protest too much when this is proposed, having two coworkers check each other's code is also a possibility.

9.7.3 Login Spoofing

In this insider attack, the perpetrator is a legitimate user who is attempting to collect other people's passwords through a technique called **login spoofing**. It is typically employed in organizations with many public computers on a LAN used by multiple users. Many universities, for example, have rooms full of computers where students can log onto any computer. It works like this. Normally, when no one is logged in on a UNIX computer, a screen similar to that of Fig. 9-32(a) is displayed. When a user sits down and types a login name, the system asks for a password. If it is correct, the user is logged in and a shell (and possibly a GUI) is started.



Figure 9-32. (a) Correct login screen. (b) Phony login screen.

Now consider this scenario. A malicious user, Mal, writes a program to display the screen of Fig. 9-32(b). It looks amazingly like the screen of Fig. 9-32(a), except that this is not the system login program running, but a phony one written by Mal. Mal now starts up the phony login program and walks away to watch the fun from a safe distance. When a user sits down and types a login name, the program responds by asking for a password and disabling echoing. After the login name and password have been collected, they are written away to a file and the phony login program sends a signal to kill its shell. This action logs Mal out and triggers the real login program to start and display the prompt of Fig. 9-32(a). The user assumes that she made a typing error and just logs in again. This time, however, it works. But in the meantime, Mal has acquired another (login name, password) pair.

9.8 OPERATING SYSTEM HARDENING

The single best way to handle security bugs is to not have them at all. Think of how nice it would be if we could accompany software with a mathematical proof that it is correct and contains no vulnerabilities. That is exactly what formal verification of software is all about. In the past, researchers have shown that it is actually